



CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

EMILLY DA SILVA DA LUZ

1- Explique com suas palavras a diferença entre backend, API e banco de dados.

O backend é a parte do sistema responsável por receber informações, aplicar regras de negócio, validar dados e coordenar o que deve acontecer dentro da aplicação. É nele, por exemplo, que podemos impedir que um horário já ocupado seja agendado novamente. A API é uma forma de comunicação com o backend. Ela funciona como uma porta de entrada pela qual outros sistemas ou aplicações podem enviar solicitações e receber respostas. Portanto, a API não é o backend inteiro, mas uma maneira de acessar as funcionalidades que ele disponibiliza. O próprio material reforça que a API é uma interface de comunicação, enquanto o backend reúne as responsabilidades que existem por trás dela. Já o banco de dados é responsável principalmente por armazenar e consultar as informações do sistema.

2- Regras que devem ser garantidas no backend

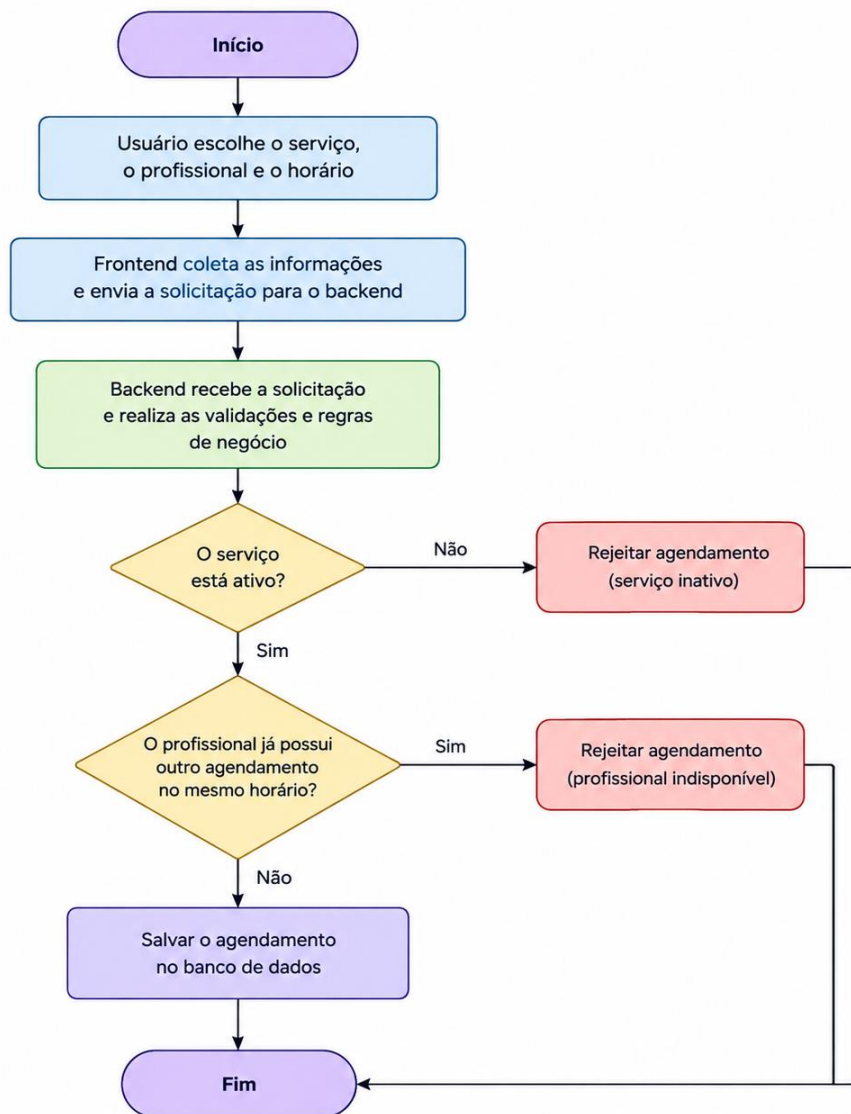
No domínio de agendamentos, o backend deve garantir que um profissional não possua dois agendamentos no mesmo horário, que serviços desativados não possam ser agendados e que todo agendamento esteja relacionado a um profissional e a um serviço válido. Essas regras devem ser verificadas no backend para que não dependam apenas da interface utilizada pelo usuário.

3- Validação sintática x regra de negócio

O e-mail sem @, o preço negativo e o CPF com quantidade incorreta de dígitos podem ser classificados como validações sintáticas, pois verificam se os dados recebidos estão dentro de um formato ou condição aceitável. Já o conflito de horário e o agendamento de um serviço desativado são regras de negócio, pois dependem das regras e do estado da aplicação para determinar se determinada ação pode ou não ser realizada.

4- Diagrama

Em um sistema de agendamento online, o processo começa quando o usuário escolhe um serviço, um profissional e um horário. O frontend coleta essas informações e envia a solicitação para o backend. O backend realiza as validações necessárias e aplica as regras de negócio, como verificar se o serviço está ativo e se o profissional já possui outro agendamento no mesmo horário. Caso todas as condições sejam atendidas, o agendamento é salvo no banco de dados.



Atividade 2 - Runtime Node.js

Questão 1

Enunciado: Explique por que `console.log` pode existir no navegador e no Node.js, embora os dois ambientes sejam diferentes.

Resposta:

JavaScript é a linguagem de programação, enquanto o navegador e o Node.js são ambientes capazes de executá-la. Ambos disponibilizam uma implementação do objeto `console`, por isso o método `console.log` pode ser utilizado nos dois ambientes.

A diferença está principalmente nos demais recursos disponíveis. O navegador possui APIs relacionadas à página e à interação com o usuário, enquanto o Node.js oferece recursos relacionados ao sistema de arquivos, processos, rede e sistema operacional.

Questão 2

Enunciado: Pesquise no REPL o resultado de `process.version` e `process.platform` e explique o que cada propriedade representa.

Resposta:

A propriedade `process.version` informa a versão do Node.js que está sendo executada.

Já a propriedade `process.platform` identifica a plataforma do sistema operacional em que o processo está sendo executado. Alguns possíveis valores são `win32`, para Windows, `linux`, para Linux, e `darwin`, para macOS.

Os valores apresentados dependem do ambiente em que os comandos forem executados.

Questão 3

Enunciado: Liste três recursos que fazem sentido no Node.js, mas não são responsabilidades típicas de uma página no navegador.

Resposta:

Entre os recursos característicos do Node.js estão o acesso ao sistema de arquivos, o acesso a processos e variáveis de ambiente e a possibilidade de criar servidores e realizar operações de rede.

Atividade 3 - Variáveis e escopo

Questão 3

Enunciado: Explique por que `const service = {}` não torna `service` imutável.

Resposta:

A declaração com `const` impede que a variável receba uma nova referência, mas não impede que as propriedades internas do objeto sejam modificadas.

Dessa forma, é possível alterar propriedades do objeto `service`. O que não é permitido é substituir completamente a referência armazenada na variável por outro objeto.

Atividade 4 - Tipos e comparações

Questão 1

Enunciado: Execute cinco comparações com == e === e explique os resultados.

Resposta:

O operador == realiza uma comparação que pode envolver conversão automática de tipos. Isso significa que valores de tipos diferentes podem ser considerados iguais após essa conversão.

O operador ===, por outro lado, compara tanto o valor quanto o tipo dos dados, sem realizar coerção. Por esse motivo, sua utilização é mais previsível e adequada para validações em aplicações.

Atividade 5 - Strings e conversões

Questão 3

Enunciado: Demonstre por que parseInt("10min", 10) e Number("10min") produzem comportamentos diferentes.

Resposta:

O parseInt tenta interpretar a parte numérica existente no início da string. Por isso, ao receber "10min", consegue identificar o valor 10.

O Number, por outro lado, tenta converter todo o conteúdo recebido em um número. Como "10min" contém caracteres que não fazem parte de uma representação numérica válida, a conversão resulta em NaN.

Atividade 6 - Operadores modernos

Questão 2

Enunciado: Compare o resultado de 0 || 20 e 0 ?? 20.

Resposta:

A expressão 0 || 20 resulta em 20, pois o operador || considera o número zero como um valor falsy e utiliza o valor seguinte.

Já a expressão 0 ?? 20 resulta em 0, pois o operador ?? somente utiliza o valor alternativo quando o primeiro valor é null ou undefined. Dessa maneira, valores como 0, false e string vazia são preservados.

Questão 3

Enunciado: Reescreva um ternário aninhado utilizando if/else e avalie qual versão é mais legível.

Resposta:

Quando existem várias condições, a estrutura if/else tende a apresentar maior legibilidade do que um conjunto de operadores ternários aninhados.

Essa estrutura deixa cada condição mais explícita, facilita a identificação dos diferentes caminhos possíveis e torna futuras alterações no código mais simples.

Atividade 8 - Funções

Questão 3

Enunciado: Explique qual das duas funções seria mais fácil de testar e por quê.

Resposta:

As duas funções são simples de testar porque recebem os dados necessários por meio de parâmetros e não dependem de variáveis externas.

A função `calculateEndTime` é especialmente simples, pois, para os mesmos valores de entrada, sempre retorna o mesmo resultado e não produz efeitos colaterais. Esse comportamento torna o teste mais previsível.

Atividade 9 - Closures e estado

Questão 2

Enunciado: Explique o que acontece com o estado quando o processo Node.js é encerrado.

Resposta:

Quando um estado é mantido apenas na memória da aplicação, ele é perdido no momento em que o processo Node.js é encerrado.

Ao iniciar a aplicação novamente, os valores armazenados anteriormente não estarão mais disponíveis, a menos que tenham sido registrados em algum mecanismo externo de persistência, como um banco de dados.

Questão 3

Enunciado: Explique por que duas instâncias do backend não compartilhariam esse contador.

Resposta:

Cada processo da aplicação possui sua própria área de memória. Por esse motivo, duas instâncias do backend manteriam contadores independentes.

Caso fosse necessário compartilhar o mesmo estado entre diferentes instâncias, seria preciso utilizar um mecanismo externo de armazenamento ou coordenação.

Atividade 12 - JSON

Questão 3

Enunciado: Explique por que funções e undefined não são representados normalmente em JSON.

Resposta:

JSON é um formato textual utilizado principalmente para armazenamento e troca de dados. Por esse motivo, possui um conjunto específico de tipos de valores que podem ser representados.

Funções representam comportamentos e não fazem parte desse formato. O valor undefined também não possui uma representação padrão em JSON. Quando uma propriedade de um objeto possui esse valor, ela normalmente é ignorada durante a conversão realizada pelo `JSON.stringify`.

Atividade 14 - Erros, exceções e Fail Fast

Questão 3

Enunciado: Dê um exemplo de erro que não deveria ser ignorado silenciosamente.

Resposta:

Um exemplo é a indisponibilidade do banco de dados. Esse tipo de falha não deve ser ignorado, pois pode impedir que a aplicação consulte ou registre informações corretamente.

A camada responsável deve registrar ou propagar o erro para que o problema possa ser identificado e tratado. Ignorar uma falha desse tipo pode esconder problemas de funcionamento e provocar inconsistências no sistema.

Atividade 16 - Process, argumentos e configuração

Questão 3

Enunciado: Explique por que `process.env.PORT` é uma string e precisa ser convertido.

Resposta:

As variáveis de ambiente são disponibilizadas ao processo como texto. Por isso, mesmo quando uma variável representa um valor numérico, como uma porta de servidor, seu conteúdo inicialmente é uma string.

Quando esse valor precisa ser utilizado como número, é necessário realizar uma conversão explícita utilizando `Number` e verificar se o resultado obtido é válido.

Atividade 18 - Validação nas fronteiras

Questão 2

Enunciado: Diferencie formato inválido de uma situação em que já existe um serviço com o mesmo nome.

Resposta:

Um formato inválido deve ser identificado durante a validação dos dados de entrada. Nessa etapa são verificados aspectos como campos obrigatórios, tipos de dados e formatos esperados.

A existência de outro serviço com o mesmo nome depende das informações que já estão armazenadas na aplicação. Portanto, essa verificação representa uma regra de negócio e deve ocorrer depois que os dados de entrada já tiverem sido considerados válidos.

Atividade 26

Erro

1

Trechos: `x`, `a`, `b`, `c`, `fazer`, `n`, `achou`.

Esses nomes não revelam a finalidade dos dados nem das funções.

Correção: usar nomes como `services`, `serviceName`, `durationMinutes`, `price`, `createService` e `findServiceByName`.

Erro 2

Trechos: `a == undefined`, `b == ""`, `x[i].id == id`.

O operador `==` permite coerção implícita de tipos e pode produzir resultados inesperados.

Correção: usar `===` e `!==`, deixando a conversão explícita quando necessária.

Erro 3

Trecho: `fazer("Consulta", "45", 150);`

A duração pode permanecer como texto e não há conversão ou validação numérica.

Correção: converter com `Number()` e validar se é um número inteiro positivo.

Erro 4

Trechos: `if (c < 0)` e verificações de strings vazias.

O código não valida preço como número, duração inteira, valores finitos ou espaços em branco.

Correção: normalizar strings com `trim()` e validar cada campo explicitamente.

Erro 5

Trecho: `ativo: "sim"`.

Usar strings como `"sim"` e `"nao"` cria estados textuais desnecessários e facilita erros de comparação.

Correção: usar `ativo: true` ou `ativo: false`.

Erro 6

Trechos: `x.push(obj)`, `contador = contador + 1`.

O array e o contador são globais e podem ser alterados por qualquer parte do arquivo.

Correção: encapsular o estado em um repositório e expor somente operações controladas.

Erro 7

Trecho: `return x;`.

Quem chama `listar()` pode alterar diretamente o estado interno.

Correção: retornar uma cópia com `[...services]` ou objetos copiados.

Erro 8

Trecho: `for (...) { if (...) { achou = x[i]; } }`.

O laço continua desnecessariamente e pode sobrescrever o resultado.

Correção: usar `find()`.

Erro 10

Trecho: `x.push(obj)`.

O sistema permite cadastrar vários serviços com o mesmo nome, inclusive diferenças apenas de maiúsculas e minúsculas.

Correção: normalizar o nome e verificar duplicidade antes do cadastro.